

OpenSearch Service

Product Search · Autocomplete · Facets · ILM · Analytics · FGAC

AWS Tutorial · ShopFlow

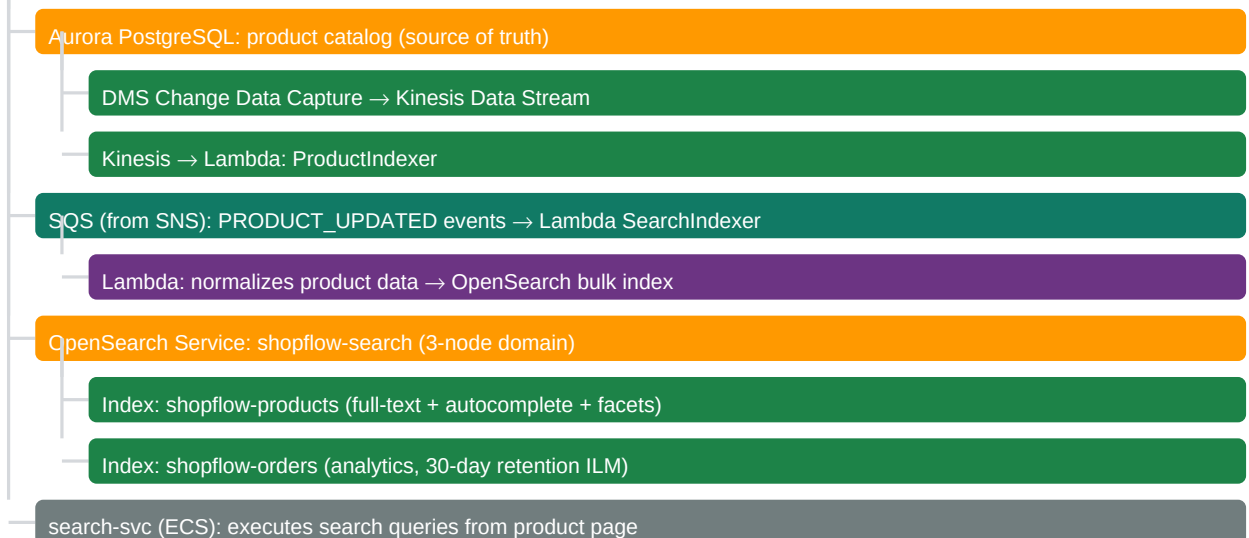
12-00

Chapter Overview — OpenSearch at ShopFlow

ShopFlow uses Amazon OpenSearch Service for product search, autocomplete, and order analytics. Aurora PostgreSQL handles transactional queries but cannot support full-text search, fuzzy matching, or faceted filtering at scale. OpenSearch ingests product data from Aurora via DynamoDB Streams + Lambda.

Use Case	OpenSearch Feature	Query Volume	Latency SLA
Product search	Full-text, fuzzy match, relevance scoring	2,000 searches/min	p99 < 200ms
Autocomplete (type-ahead)	Completion suggester, prefix queries	10,000 req/min	p99 < 50ms (fast typing)
Faceted filter	Aggregations (brand, price range, rating)	1,000 req/min	p99 < 300ms
Order analytics	Date histogram, sum aggregations	100 dashboards/min	p99 < 2s (dashboard load)
Synonym search	Synonym analyzer (TV → television)	Included in product search	Same as search

ShopFlow OpenSearch Data Pipeline



12-01

OpenSearch Domain CDK — 3-Node Cluster

ShopFlow OpenSearch domain: 3 dedicated master nodes (m6g.large) + 3 data nodes (r6g.large.search, 100GB EBS each). Multi-AZ with zone awareness across 3 AZs. Master nodes handle cluster state; data nodes handle indexing and search. Graviton2 (r6g) saves 20% vs x86.

OpenSearch CDK

```
const osDomain = new opensearch.Domain(this, "ShopFlowSearch", {
  domainName: "shopflow-search",
  version: opensearch.EngineVersion.OPENSEARCH_2_11,
  capacity: {
    masterNodes: 3,
```

```
masterNodeInstanceType: "m6g.large.search", // Graviton2 master
dataNodes: 3,
dataNodeInstanceType: "r6g.large.search", // Graviton2 data node
},
ebs: {
volumeSize: 100, // 100 GB gp3 per data node = 300GB total
volumeType: ec2.EbsDeviceVolumeType.GP3,
iops: 3000,
},
zoneAwareness: {
enabled: true,
availabilityZoneCount: 3, // 1 data node per AZ
},
vpc,
vpcSubnets: [{subnetType: ec2.SubnetType.PRIVATE_ISOLATED}],
securityGroups: [sgOpenSearch],
encryptionAtRest: { enabled: true },
nodeToNodeEncryption: true,
enforceHttps: true,
fineGrainedAccessControl: {
masterUserName: "shopflow-admin",
masterUserPassword: masterPwSecret.secretValue,
},
logging: {
slowSearchLogEnabled: true,
slowIndexLogEnabled: true,
appLogEnabled: true,
},
});
```

Component	Count	Instance Type	Purpose
Dedicated master	3 (odd number)	m6g.large.search (2 vCPU, 8 GB RAM)	Cluster state management; no data stored
Data nodes	3 (1 per AZ)	r6g.large.search (2 vCPU, 16 GB RAM)	Indexing + search; 100GB EBS each
Storage total	300 GB	gp3 EBS	40M products × 7KB avg doc = 280GB; 20GB headroom
Replication	1 primary + 1 replica	Data spread across 3 nodes	Any 1 node loss = no data loss (replica on another AZ)

12-02

Index Mapping — Product Schema Design

OpenSearch index mappings define field types (text, keyword, float, date). text fields are analyzed (tokenized, stemmed) for full-text search. keyword fields are not analyzed — used for exact match, aggregations (facets), and sorting. Choosing text vs keyword correctly is critical for search quality and performance.

Product Index Mapping

```
PUT /shopflow-products
{
  "settings": {
    "number_of_shards": 3, // 1 primary shard per data node
    "number_of_replicas": 1, // 1 replica = 2x storage, 0 data loss on 1 node failure
    "analysis": {
      "analyzer": {
        "shopflow_analyzer": {
          "type": "custom",
          "tokenizer": "standard",
          "filter": ["lowercase","stop","snowball", // stemming: running → run
                    "shopflow_synonyms"] // TV → television
        }
      }
    },
    "filter": {
```

```

"shopflow_synonyms": {
  "type": "synonym",
  "synonyms": ["tv, television", "laptop, notebook", "sneakers, trainers"]
}
},
"mappings": {
  "properties": {
    "productId": {"type": "keyword"}, // exact match, no analysis
    "name": {"type": "text", "analyzer": "shopflow_analyzer",
    "fields": {"raw": {"type": "keyword"}}}, // keyword sub-field for sort
    "description": {"type": "text", "analyzer": "shopflow_analyzer"},
    "brand": {"type": "keyword"}, // facet (aggregation)
    "category": {"type": "keyword"}, // facet
    "price": {"type": "float"}, // range query + sort
    "rating": {"type": "float"}, // sort
    "stockCount": {"type": "integer"},
    "imageUrl": {"type": "keyword", "index": false}, // stored not indexed
    "suggest": {"type": "completion"}, // autocomplete
    "updatedAt": {"type": "date", "format": "strict_date_time"}
  }
}
}

```

■ Use multi-fields to index the same content two ways: name as text (analyzed, searchable) and name.raw as keyword (not analyzed, sortable/aggregatable). This avoids maintaining two separate fields. Query name for full-text search; sort or aggregate on name.raw. Without .raw, sorting on a text field returns inconsistent results because analyzed fields split into tokens.

12-03

Bulk Indexing — Lambda SearchIndexer

Lambda SearchIndexer receives PRODUCT_UPDATED events from SQS and indexes products into OpenSearch using the bulk API. Bulk indexing 1,000 products in one API call is 1,000× more efficient than individual index calls. OpenSearch Java client (opensearch-java) handles connection pooling and retry.

SearchIndexer Lambda — Bulk Index

```

@Component
public class SearchIndexerHandler
implements RequestHandler<SQSEvent, SQSBatchResponse> {
  private final OpenSearchClient osClient;
  private static final String INDEX = "shopflow-products";

  @Override
  public SQSBatchResponse handleRequest(SQSEvent event, Context ctx) {
    List<BulkOperation> ops = new ArrayList<>();
    List<SQSBatchResponse.BatchItemFailure> failures = new ArrayList<>();
    for (SQSEvent.SQSMessage msg : event.getRecords()) {
      ProductDoc doc = json.readValue(msg.getBody(), ProductDoc.class);
      ops.add(new BulkOperation.Builder()
        .index(idx -> idx.index(INDEX))
        .id(doc.getProductId())
        .document(doc)
        .build());
    }
    // Single bulk API call for up to 10 products (SQS batch size)
    BulkResponse resp = osClient.bulk(b -> b.index(INDEX).operations(ops));
    // Report per-item failures (partial batch success)
    for (int i = 0; i < resp.items().size(); i++) {
      if (resp.items().get(i).error() != null) {
        failures.add(SQSBatchResponse.BatchItemFailure.builder()
          .withItemIdentifier(event.getRecords().get(i).getMessageId()).build());
      }
    }
  }
}

```

```

}
}
return SQSBatchResponse.builder().withBatchItemFailures(failures).build();
}
}

```

■ OpenSearch bulk API accepts up to 10MB per request (default). Each product document ~7KB; SQS batch of 10 = 70KB — well within limits. For full catalog re-indexing (40M products), use batches of 500-1,000 documents sent from a Fargate batch task (not Lambda — 15-min timeout limit). Full catalog index: 40M products × 7KB / 500-doc batches = 80K bulk requests; at 100 batches/s from 4 Fargate tasks: 13 minutes.

12-04

Full-Text Search Query — Relevance & Boosting

OpenSearch multi_match query searches across multiple fields with per-field boost factors. Name matches are boosted 3× over description matches. Function score applies a logarithmic boost based on product rating. Bool query combines full-text with exact filters for faceted search.

Product Search Query

```

// POST /shopflow-products/_search
{
  "query": {
    "function_score": {
      "query": {
        "bool": {
          "must": [
            {
              "multi_match": {
                "query": "wireless bluetooth headphones",
                "fields": ["name^3", "brand^2", "description^1"],
                "type": "best_fields",
                "fuzziness": "AUTO" // handles typos: headphone → headphones
              }
            }
          ],
          "filter": [
            {"term": {"category": "electronics"}},
            {"range": {"price": {"gte": 20, "lte": 300}}},
            {"range": {"stockCount": {"gt": 0}}} // in-stock only
          ]
        }
      },
      "functions": [
        {
          "field_value_factor": {
            "field": "rating",
            "factor": 1.2,
            "modifier": "log1p", // log(1 + rating): diminishing returns
            "missing": 1 // default for products without rating
          }
        }
      ],
      "score_mode": "multiply"
    }
  },
  "aggregations": {
    "brands": {"terms": {"field": "brand", "size": 20}},
    "price_ranges": {"range": {"field": "price",
      "ranges": [{"to": 50}, {"from": 50, "to": 150}, {"from": 150}]}},
    "avg_rating": {"avg": {"field": "rating"}}
  },
  "size": 20, "from": 0
}

```

```
}
```

12-05

Autocomplete — Completion Suggester

OpenSearch Completion Suggester provides sub-50ms autocomplete by storing prefix trees in memory (not on disk). ShopFlow indexes product names with weight (sales rank) so popular products appear first. The suggest API returns top-10 completions for any prefix.

Autocomplete Indexing + Query

```
// Index: suggest field with input phrases + weight
"suggest": {
  "input": ["Sony WH-1000XM5 Wireless", "Sony Wireless", "Wireless Headphones"],
  "weight": 950 // sales rank score – higher = shown first in suggestions
}

// Query: prefix "wire" → top 5 suggestions
POST /shopflow-products/_search
{
  "suggest": {
    "product_suggest": {
      "prefix": "wire",
      "completion": {
        "field": "suggest",
        "size": 10,
        "fuzzy": { "fuzziness": 1 }, // typo tolerance
        "contexts": {
          "category": [{"context": "electronics"}] // filter by active category
        }
      }
    }
  }
}
```

Suggestion Method	Latency	Memory Use	Fuzzy Support	Weight/Ranking
Completion Suggester (ShopFlow)	20-50ms — in-memory prefix tree	High — all suggestions in JVM (heap)	Yes (fuzziness param)	Yes — weight field
Match Phrase Prefix query	10-50ms — disk-based	Normal (no extra memory)	No	By relevance score only
Search As You Type field type	10-30ms — special index structure	Medium	Limited	By score
Edge n-gram tokenizer	20-100ms — standard text analysis	Normal	No	By score

■ Completion Suggester stores all input strings in memory on each data node. For 40M products with 3 suggest inputs each = 120M strings × ~20 bytes avg = 2.4GB per node. Monitor JVMMemoryPressure metric — above 75%, heap GC pauses increase sharply. Limit suggest input count to 2-3 most important phrases per product. Use Search As You Type as an alternative if memory is a concern.

12-06

Index Lifecycle Management — Order Analytics

ILM automatically transitions indices through phases (hot → warm → cold → delete) based on age and size. ShopFlow order analytics use daily indices with 30-day retention. Hot phase: 3 replicas on r6g nodes. Cold phase: data moved to UltraWarm storage (S3-backed) at 90% cost reduction.

ILM Policy CDK — Order Analytics 30-day Retention

```
PUT /_plugins/_ism/policies/shopflow-orders-ism
{
  "policy": {
    "description": "30-day order analytics lifecycle",
    "default_state": "hot",
    "states": [
      {

```

```
"name": "hot",
"transitions": [{"state_name": "warm",
"conditions": {"min_index_age": "7d"}}]
},
{
"name": "warm",
"actions": [{"replica_count": {"number_of_replicas": 0}}],
"transitions": [{"state_name": "delete",
"conditions": {"min_index_age": "30d"}}]
},
{"name": "delete", "actions": [{"delete": {}}]}
],
"ism_template": [{"index_patterns": ["shopflow-orders-*"],
"priority": 100}]
}
}
```

ILM Phase	Trigger	Replica Count	Storage Cost	ShopFlow Config
Hot (active writes+reads)	Day 0 (creation)	1 replica (2 copies)	\$0.135/GB-mo (gp3)	Days 0-7: all write + read queries
Warm (reads only)	Age >= 7 days	0 replicas (1 copy)	\$0.135/GB-mo	Days 7-30: analytics queries only
Delete	Age >= 30 days	—	\$0	Day 30+: auto-deleted
UltraWarm (optional)	Age >= 7 days on domain replicas	Overlaps Warm	\$0.024/GB-mo (S3)	Not configured — 30-day delete suffices

■ UltraWarm nodes store index data on S3 (S3-backed cache) at \$0.024/GB vs \$0.135/GB for hot storage — 82% cheaper. Use UltraWarm for indices that are queried infrequently but must be retained for compliance (e.g., order history: 90-day hot, 7-year cold/UltraWarm). ShopFlow order analytics only need 30 days — hot → delete suffices without UltraWarm.

12-07

Fine-Grained Access Control & Security

OpenSearch fine-grained access control (FGAC) restricts data access at index, field, and document level. ShopFlow uses FGAC roles: search-svc read-only role (only GET /shopflow-products/_search), indexer-lambda write role (only bulk index), and admin role for cluster management.

Role	Cluster Actions	Index Permissions	Document/Field Restriction
search_reader	No cluster perms	indices:data/read/search, indices:data/read/msearch	None (shopflow-products fields readable)
indexer_writer	No cluster perms	indices:data/write/bulk on shopflow-products, shopflow-orders	Writes all fields
admin	cluster:monitor/*	All on all indices	None (admin)
dashboard_user	No cluster perms	indices:data/read on shopflow-orders-*	Exclude customerEmail field (PII)

Fine-Grained Access — Java REST Client Config

```
// OpenSearch client: FGAC with IAM SigV4 signing
SdkHttpClient httpClient = ApacheHttpClient.builder().build();
Aws4Signer signer = Aws4Signer.create();
OpenSearchTransport transport = new AwsSdk2Transport(
    httpClient,
    HttpHost.create(domain.attrDomainEndpoint()),
    "es", // service name for SigV4 signing
    Region.US_EAST_1,
    AwsSdk2TransportOptions.builder()
        .setCredentials(DefaultCredentialsProvider.create())
        .build());
OpenSearchClient client = new OpenSearchClient(transport);
```

■ Use IAM SigV4 authentication for OpenSearch (not username/password) from Lambda and ECS tasks. The task/execution role signs requests with its temporary credentials — no stored passwords. SigV4 credentials rotate automatically every 15 minutes (STS temporary credentials). Map the IAM ARN to an OpenSearch FGAC role: Settings → Security → Roles → Mapped Users tab in OpenSearch Dashboards.

12-08

OpenSearch Dashboards — Order Analytics

OpenSearch Dashboards (formerly Kibana) provides visualization for the shopflow-orders index. ShopFlow uses Dashboards for: daily revenue trend, top-selling products, order fulfillment lag histogram, and geographic sales heat map. Dashboards accessed via VPC + Cognito authentication.

Dashboard	Visualization Type	Data Source	Refresh Rate
Daily Revenue Trend	Date histogram sum(totalAmt)	shopflow-orders-*	5 minutes
Top Products by Revenue	Terms aggregation + bar chart	shopflow-orders-*	15 minutes
Order Fulfillment Time	Histogram (fulfillmentMs)	shopflow-orders-*	5 minutes
Geographic Sales Map	Geo-point heatmap (shippingState)	shopflow-orders-*	1 hour
Real-time Search Queries	Terms aggregation (searchQuery)	shopflow-search-logs-*	1 minute
Search Zero-Results Rate	Filter aggregation (hits.total=0)	shopflow-search-logs-*	5 minutes

Search Analytics — Log Every Query

```
// search-svc: log every search query to OpenSearch for analytics
@Component
public class SearchAnalyticsLogger {
    @Async
    public void logSearch(String query, int resultsCount, long latencyMs) {
        String today = LocalDate.now().format(DateTimeFormatter.ISO_LOCAL_DATE);
        String index = "shopflow-search-logs-" + today;
        Map<String,Object> doc = Map.of(
            "query", query,
            "resultsCount", resultsCount,
            "latencyMs", latencyMs,
            "zeroResults", resultsCount == 0,
            "@timestamp", Instant.now().toString());
        osClient.index(i -> i.index(index).document(doc));
    }
}
```

12-09

Monitoring & Slow Query Tuning

Key OpenSearch metrics: SearchLatency (p99), IndexingLatency, JVMMemoryPressure, CPUUtilization, FreeStorageSpace, and KibanaHealthyNodes. Slow query logs capture queries exceeding thresholds — used to identify missing indices, over-broad queries, and aggregation performance issues.

Metric	Namespace	Alert Threshold	Action
SearchLatency p99	AWS/ES	> 200ms for 5 min	Check slow query log; add index or optimize query
JVMMemoryPressure	AWS/ES	> 85% for 5 min	Scale up data nodes or reduce heap-heavy operations
FreeStorageSpace	AWS/ES	< 20% (60GB remaining)	Add data nodes or increase EBS volume (online expansion)
CPUUtilization	AWS/ES	> 80% for 15 min	Scale out data nodes; check for expensive aggregations
KibanaHealthyNodes	AWS/ES	< 3	OpenSearch cluster health degraded — check CloudWatch Logs

Enable Slow Query Logging

```
// Enable via OpenSearch API (set threshold):
PUT /shopflow-products/_settings
{
  "index.search.slowlog.threshold.query.warn": "200ms",
  "index.search.slowlog.threshold.query.info": "100ms",
  "index.search.slowlog.threshold.fetch.warn": "50ms",
  "index.search.slowlog.threshold.fetch.info": "25ms"
}
// Slow queries → CloudWatch Logs:
/aws/OpenSearchService/domains/shopflow-search/search-slowlogs
// Analyze: CloudWatch Logs Insights query:
// fields @message | filter @message like /took:/ | parse @message
// "took[*]:*" as took | stats avg(took), max(took) by bin(5min)
```

■ Zero-results-rate is the most actionable search quality metric. If 15% of searches return no results, investigate the top zero-result queries in OpenSearch Dashboards. Common causes: (1) missing synonyms (add to synonym filter), (2) typos not handled (increase fuzziness), (3) category mismatch (user searches in wrong category), (4) products not indexed (check indexer DLQ). Target: zero-results rate <5%.

12-10

Scaling & Shard Strategy

OpenSearch shard sizing: each shard should be 10-50GB. Too many shards overload the cluster with overhead; too few create large shards that are slow to rebalance. ShopFlow product index: 280GB total, 3 primary shards (93GB each) + 1 replica each = 6 shards across 3 data nodes.

Shard Config	Total Shards	Per-Node Load	Rebalance Time	Recommendation
1 primary shard (default)	2 (1P + 1R)	Both on same node if not 1 shard	1 shard = full rebalance on node change	Bad — no parallelism; full rebalance on s
3 primary shards (ShopFlow)	6 (3P + 3R)	2 shards per node	33% rebalance when adding 1 node	Good — matches node count; even dis
10 primary shards	20 (10P + 10R)	6.7 shards per node	10% rebalance per new node	Over-sharded — overhead without bene

Rolling Index Strategy for Order Analytics

```
// Create daily index: shopflow-orders-YYYY-MM-DD
// Index alias: shopflow-orders (points to latest index for writes)
// Queries: search shopflow-orders-* (all indices via wildcard)
// Create alias on new daily index:
POST /_aliases
{
  "actions": [
    {"remove": {"index": "shopflow-orders-2024-11-28", "alias": "shopflow-orders"}},
    {"add": {"index": "shopflow-orders-2024-11-29", "alias": "shopflow-orders",
      "is_write_index": true}}
  ]
}
// Lambda: EventBridge Scheduler at midnight creates new daily index + rotates alias
```

■ OpenSearch supports online storage expansion: increase EBS volume size without downtime or shard rebalancing. When FreeStorageSpace <20%: update the CDK ebs.volumeSize to a larger value and redeploy — OpenSearch handles the resize online. For data node count changes (add 4th node): OpenSearch automatically rebalances shards across nodes (background operation; <1 hour for 300GB).

12-11

Cost Optimization — Right-Sizing & UltraWarm

OpenSearch cost: \$0.128/hr per r6g.large.search (data node) + \$0.045/hr per m6g.large.search (master node). Monthly: 3 data nodes × \$0.128 × 720 = \$276 + 3 master nodes × \$0.045 × 720 = \$97 = \$373/month. EBS: 300GB × \$0.135/GB = \$40.50/month. Total: ~\$414/month.

Cost Component	Amount	Unit Price	Monthly Cost
Data nodes: 3× r6g.large.search	3 × 720 hr	\$0.128/hr	\$276.48
Master nodes: 3× m6g.large.search	3 × 720 hr	\$0.045/hr	\$97.20
EBS gp3: 300 GB (3 nodes × 100GB)	300 GB	\$0.135/GB	\$40.50
OpenSearch Dashboards	Included	\$0	\$0
Total monthly			\$414.18
With 1-year Reserved (35% off data+master)			\$280/month (-\$134/mo)

12-10

Kinesis Firehose to OpenSearch Ingest Pipeline

Kinesis Data Firehose delivers streaming data directly to OpenSearch without Lambda. ShopFlow uses Firehose to stream order events (from Kinesis Data Streams) into the shopflow-orders index. Firehose handles retry, buffering (1MB or 60s), and transformation via Lambda before delivery.

Firehose CDK Order Events to OpenSearch

```
const orderFirehose = new firehose.CfnDeliveryStream(this, "OrderFirehose", {
  deliveryStreamName: "shopflow-orders-os",
  deliveryStreamType: "KinesisStreamAsSource",
  kinesisStreamSourceConfiguration: {
    kinesisStreamArn: orderStream.streamArn,
    roleArn: firehoseRole.roleArn,
  },
  amazonopensearchserviceDestinationConfiguration: {
    domainArn: osDomain.domainArn,
    indexName: "shopflow-orders",
    indexRotationPeriod: "OneDay",
    bufferingHints: {
      intervalInSeconds: 60,
      sizeInMBs: 1,
    },
    retryOptions: { durationInSeconds: 300 },
    s3BackupMode: "FailedDocumentsOnly",
    s3Configuration: {
      bucketArn: backupBucket.bucketArn,
      roleArn: firehoseRole.roleArn,
      prefix: "failed-orders/",
    },
  },
});
```

Firehose Feature	ShopFlow Config	Purpose
Source	Kinesis Data Streams (orderStream)	Reads order events Kinesis stream
Index rotation	OneDay	Daily index shopflow-orders-YYYY-MM-DD auto-created
Buffer	60s or 1MB	Micro-batch delivery - reduces OpenSearch bulk call overhead
Retry	300 seconds	Handles transient OpenSearch unavailability
S3 backup	FailedDocumentsOnly	Failed index requests saved to S3 for reprocessing

12-11

Zero-Downtime Reindex with Aliases

OpenSearch index mappings are immutable: field types cannot be changed after creation. Adding a new analyzed field requires a full reindex. Aliases allow zero-downtime reindex: new index created in background, alias atomically

switched after completion. ShopFlow uses this for mapping migrations.

Zero-Downtime Reindex Pattern

```
// Step 1: Create v2 index with updated mapping
PUT /shopflow-products-v2
{ "settings": {...same as v1...},
  "mappings": { "properties": {
    // all v1 fields plus:
    "tags": {"type": "keyword"} // NEW field
  }}}
// Step 2: Reindex v1 to v2 (background operation)
POST /_reindex
{
  "source": {"index": "shopflow-products-v1"},
  "dest": {"index": "shopflow-products-v2", "op_type": "create"},
  "conflicts": "proceed"
}
// Step 3: Atomic alias switch
POST /_aliases
{
  "actions": [
    {"remove": {"index": "shopflow-products-v1", "alias": "shopflow-products"}},
    {"add": {"index": "shopflow-products-v2", "alias": "shopflow-products",
      "is_write_index": true}}
  ]
}
```

■ During reindex, new writes go to v1 (alias still points there). After alias switch, new writes go to v2. For 40M products at 10K docs/s reindex rate: ~67 minutes. To avoid missing writes during this window, replay events from an SQS queue or reindex with a date filter after alias switch.

12-12

Aggregations and Faceted Navigation Deep Dive

Faceted navigation lets users filter search results by brand, price, rating, and category. OpenSearch aggregations compute facet counts in a single query alongside search results. ShopFlow runs terms, range, and stats aggregations in every product search request.

Faceted Search Query Full Example

```
POST /shopflow-products/_search
{
  "query": {"bool": {
    "must": [{"match": {"name": "headphones"}}],
    "filter": [{"term": {"category": "electronics"}},
    {"range": {"price": {"gte": 20, "lte": 300}}}],
    "aggregations": {
      "brands": {
        "terms": {"field": "brand", "size": 20,
          "order": {"_count": "desc"}}
      },
      "price_histogram": {
        "histogram": {"field": "price", "interval": 50, "min_doc_count": 1}
      },
      "avg_rating": {"avg": {"field": "rating"}},
      "in_stock_count": {"filter": {"range": {"stockCount": {"gt": 0}}}},
      "price_stats": {"stats": {"field": "price"}}
    }
  },
  "post_filter": {"term": {"brand": "Sony"}},
  "size": 20,
  "sort": [{"_score": "desc"}, {"rating": "desc"}]
}
```

■ `post_filter` applies AFTER aggregation so facet counts reflect all results before brand filter, but displayed products are filtered. This is standard e-commerce behavior: selecting Sony filter keeps all brand counts visible. Without `post_filter`, selecting Sony would narrow aggregation counts too, hiding other brand options.

12-13

OpenSearch Snapshots and Backup Strategy

OpenSearch automated snapshots (free, hourly) retain up to 14 days. Manual snapshots stored in S3 (user-owned bucket) have indefinite retention. ShopFlow takes manual snapshots daily before index schema changes, and weekly for disaster recovery.

Snapshot Type	Frequency	Retention	Cost	Use Case
Automated (AWS managed)	Hourly	14 days	Free	Hourly restore point; same-region only
Manual (S3-based)	Daily	Indefinite	S3 storage (\$0.023/GB-mo)	Pre-change backup; cross-region DR restore
Cross-region manual	Weekly	90 days	S3 cross-region transfer + storage	Business continuity: restore to us-west-2

Manual Snapshot Lambda

```
// Register S3 snapshot repository (one-time setup):
PUT /_snapshot/shopflow-backup
{
  "type": "s3",
  "settings": {
    "bucket": "shopflow-os-snapshots",
    "region": "us-east-1",
    "role_arn": "arn:aws:iam::ACCOUNT:role/OpenSearchSnapshotRole"
  }
}

// Take snapshot (EventBridge Scheduler Lambda daily at 02:00 UTC):
PUT /_snapshot/shopflow-backup/snapshot-2024-11-29
{
  "indices": "shopflow-products,shopflow-orders-*",
  "include_global_state": false
}
```

■ OpenSearch automated snapshots are SAME-REGION. If us-east-1 fails, automated snapshots are inaccessible. For multi-region DR: copy manual snapshots to a second S3 bucket in us-west-2 using S3 Cross-Region Replication. RTO: ~2 hours. RPO: 24 hours (daily manual) or 1 hour (automated, same-region).

12-14

Search Performance Tuning and Caching

OpenSearch caching layers: request cache (aggregation results per shard), query cache (frequently used filters), and field data cache (aggregations on text fields — avoid). ShopFlow enables request cache for all search queries. ElastiCache Redis in front of OpenSearch caches popular search results for 5 minutes.

Two-Level Search Cache Strategy

```
// search-svc: Redis L1 cache in front of OpenSearch
@Service
public class CachingSearchService {
  @Cacheable(value="search", key="#query + #filters + #page",
    unless="#result.totalHits == 0")
  public SearchResponse search(String query, Map filters, int page) {
    // Cache miss: execute OpenSearch query
    return osClient.search(buildQuery(query, filters, page), ProductDoc.class);
  }
}

// Redis cache TTL: 5 minutes for non-personalized search
// Cache hit rate target: 40% (popular queries: "headphones", "laptop")
// Cache key includes: query string + active filters + page number
```

// Cache MISS on: zero-results queries (unless attribute prevents caching)

Cache Layer	Scope	TTL	Hit Rate Target	Benefit
Redis L1 (search-svc)	Full search response	5 minutes	40%	Eliminates OpenSearch query for repeated queries
OS Request Cache	Aggregation results per shard	Until index refresh (1s)	Automatic	Reuses agg computation for identical queries
OS Filter Cache	Frequently used filter results	Until index refresh	Automatic	Cached filter bitsets (category:electronics)
OS Fielddata Cache	Text field aggregations (avoid)	LRU eviction	N/A — avoid	Avoid: use keyword sub-field for aggregations

■ Enable request caching per query: set `request_cache=true` in query parameters. OpenSearch caches shard-level aggregation results — identical aggregation queries return cached results until index refresh (default: 1 second). For ShopFlow faceted search: the brand terms aggregation for `category=electronics` is cached and reused across all searches in that category, not just identical full queries.

12-15

Relevance Tuning and A/B Testing

Relevance tuning is iterative: define metrics (click-through rate, purchase rate per search), run A/B tests with different boost factors and ranking functions, measure lift. ShopFlow uses Ranking Evaluation API to compare ranking models against a gold standard query-document judgments file.

Ranking Evaluation API

```
POST /shopflow-products/_rank_eval
{
  "requests": [
    {
      "id": "bluetooth-headphones-query",
      "request": {
        "query": {"multi_match": {"query": "bluetooth headphones",
          "fields": ["name^3", "brand^2", "description"]}}
      },
      "ratings": [
        {"_id": "PROD-001", "rating": 3}, // 3 = highly relevant
        {"_id": "PROD-002", "rating": 2}, // 2 = relevant
        {"_id": "PROD-003", "rating": 0} // 0 = not relevant
      ]
    }
  ],
  "metric": {
    "ndcg": {"k": 10} // Normalized Discounted Cumulative Gain at top 10
  }
}
// Response: ndcg score 0.0-1.0; 1.0 = perfect ranking
// Tune boost factors until NDCG improves; compare baseline vs candidate
```

Relevance Metric	Description	ShopFlow Target	How to Improve
NDCG@10	Quality of top 10 results	>0.80	Tune field boosts; add <code>function_score</code>
Zero-results rate	Queries with 0 results	<5%	Add synonyms; increase fuzziness
Click-through rate	% of searches leading to product click	>30%	Improve top-3 relevance; add images to snippets
Search-to-purchase	% of searches leading to purchase	>5%	Boost highly-purchased products; personalize results
Search latency p99	99th percentile query time	<200ms	Add replicas; tune shard count; Redis cache

12-16

VPC Endpoint and Security Groups for OpenSearch

OpenSearch domain is deployed inside the ShopFlow VPC in private isolated subnets. A VPC endpoint allows ECS tasks and Lambdas to communicate with OpenSearch without traversing the public internet. Security groups restrict

access: only search-svc ECS security group and indexer Lambda security group can connect on port 443.

OpenSearch Security Group CDK

```
// Security group for OpenSearch domain
const sgOpenSearch = new ec2.SecurityGroup(this,"SgOpenSearch",{
  vpc,
  description: "OpenSearch domain – allow 443 from app tier only",
  allowAllOutbound: false,
});
// Allow search-svc ECS tasks to query OpenSearch
sgOpenSearch.addIngressRule(
  sgSearchSvc,
  ec2.Port.tcp(443),
  "search-svc ECS task HTTPS to OpenSearch"
);
// Allow Lambda indexer to bulk-index products
sgOpenSearch.addIngressRule(
  sgIndexerLambda,
  ec2.Port.tcp(443),
  "SearchIndexer Lambda HTTPS to OpenSearch"
);
// Restrict OpenSearch outbound (no outbound needed – client initiates all calls)
sgOpenSearch.addEgressRule(ec2.Peer.anyIpv4(), ec2.Port.tcp(443),
  "HTTPS egress for OpenSearch Dashboards SAML auth");
```

Component	Security Group	Allowed Port	Direction	Purpose
search-svc ECS	sgSearchSvc	443 to sgOpenSearch	Outbound	Full-text search queries
SearchIndexer Lambda	sgIndexerLambda	443 to sgOpenSearch	Outbound	Bulk product indexing
Firehose (managed)	AWS-managed Firehose SG	443 to sgOpenSearch	Outbound	Order event streaming ingest
OpenSearch Dashboards	sgOpenSearch	443 from VPN/bastion	Inbound	Admin analytics access
Bastion host	sgBastion	443 to sgOpenSearch	Outbound	Ops team debug access

■ Never place OpenSearch in public subnets. Once deployed inside a VPC, the domain gets private IP addresses in your subnets. All API calls use private IPs — no traffic leaves the VPC boundary. For OpenSearch Dashboards access by the analytics team: use AWS Client VPN or a bastion host rather than making the domain internet-accessible. Fine-grained access control adds user-level authentication on top of network-level security groups.

12-17

Black Friday Pre-Scaling and Load Testing

Black Friday search volume: 10,000 req/s (10x normal). OpenSearch must be pre-scaled 48 hours before Black Friday since node additions + shard rebalancing takes 1-2 hours. ShopFlow pre-scales from 3 to 6 data nodes and increases replicas from 1 to 2, tripling read throughput.

Metric	Normal Load	Black Friday Target	Pre-Scale Action
Search requests/s	200 req/s	2,000 req/s (10x)	6 data nodes + 2 replicas = 3x read capacity
Index requests/s	500 docs/s	1,000 docs/s (2x — more nodes)	4 nodes → 8 nodes (shard count: 4 → 8 (Firehose auto-scales))
Autocomplete req/s	1,000 req/s	10,000 req/s	Completion suggester in-memory: no separate scaling needed
Data nodes	3 (r6g.large)	6 (r6g.large)	Add 3 nodes 48hr before event; rebalance ~1hr
Replicas per shard	1	2	POST /shopflow-products/_settings: {replicas:2}

Pre-Scale Script EventBridge Scheduled Lambda

```
// Lambda triggered 48 hours before Black Friday (EventBridge Scheduler):
public void preScaleForBlackFriday() {
```

```
// Step 1: Add 3 more data nodes via AWS SDK
UpdateDomainConfigRequest req = UpdateDomainConfigRequest.builder()
    .domainName("shopflow-search")
    .clusterConfig(ClusterConfig.builder()
        .instanceCount(6) // up from 3
        .instanceType(OpenSearchPartitionInstanceType.R6G_LARGE_SEARCH)
        .build())
    .build();
osClient.updateDomainConfig(req);
// Step 2: Increase replicas (after nodes are ready ~1hr later)
// PUT /shopflow-products/_settings {"number_of_replicas": 2}
// Monitor: ClusterStatus.green before proceeding to replica increase
}
```

■ OpenSearch vertical scaling (larger instance type) requires a blue-green deployment: AWS spins up new nodes with larger instances, migrates shards, then decommissions old nodes. Takes 2-4 hours and requires maintenance window for some configurations. For Black Friday: prefer horizontal scaling (more nodes, same instance type) — online operation, no maintenance window, 1-2 hours for shard rebalancing.

12-BP

Best Practices & Anti-Patterns

- ✓ Use 3 dedicated master nodes (odd number) — prevent split-brain; masters do not hold data
- ✓ Size shards 10-50GB — too large = slow rebalance; too many = overhead; match primary count to data nodes
- ✓ Use keyword sub-field (name.raw) for sorting and aggregations on text fields
- ✓ Enable synonym filter for common e-commerce synonyms (TV → television, laptop → notebook)
- ✓ Use Completion Suggester for autocomplete — sub-5ms from in-memory prefix trees
- ✓ Index via bulk API — 500-1,000 documents per bulk call; never one document per request
- ✓ Implement ILM policies for time-series indices — auto-delete prevents unbounded storage growth
- ✓ Enable slow query logging — threshold 200ms; investigate zero-results rate weekly
- ✓ Use IAM SigV4 signing for all OpenSearch API calls from Lambda and ECS
- ✓ Monitor JVMMemoryPressure and alert at 85% — above 90%, GC pauses degrade search latency
- ✓ Set number_of_replicas=1 for production — 0 replicas = data loss on single node failure
- ✓ Use function_score with field_value_factor for rating boost — log1p prevents high-rating domination
- ✓ Log all search queries with results count to shopflow-search-logs — enables zero-results analysis
- Never use a wildcard query on unbounded fields (*) — scans all shards; causes cluster-wide CPU spike
- Do not use _update_by_query in production — lock conflicts cause IndexingLatency spikes
- Avoid more than 1,000 unique field names per index — causes mapping explosion, high heap use
- Do not expose OpenSearch Dashboards without authentication — use Cognito + SAML or FGAC
- Never delete and recreate a production index — use alias + new index + reindex for zero-downtime reindex

12-QR

Quick Reference & Interview Q&A

Concept	Key Value / Rule
text vs keyword	text: analyzed (tokenized, stemmed) for full-text search. keyword: not analyzed, for exact match, aggregat
Shard sizing	Target 10-50GB per shard. Too many shards = overhead. Match primary shards to data node count for ev
Completion Suggester	In-memory prefix tree. Sub-5ms latency. Input: array of strings. Weight: controls ranking. Context: filter by
fuzziness AUTO	AUTO: fuzziness 0 for <3 chars, 1 for 4-6 chars, 2 for >6 chars. Handles common typos. Use in multi
ILM phases	hot (write + read) → warm (read only, drop replicas) → cold (UltraWarm S3-backed) → delete. Trigger: inc
JVM heap recommendation	50% of node RAM (r6g.large = 16GB RAM → 8GB heap). Above 50%: OS page cache starved; below 50%

Concept	Key Value / Rule
Number of replicas	Production: 1 replica (2 copies per shard). Zero replicas: no data loss protection. 2 replicas: 3 copies — for

Top Interview Questions

Question	Answer
OpenSearch vs Aurora for search?	Aurora: relational queries, JOINS, ACID transactions, LIKE queries up to ~100K rows. OpenSearch: full-text
Explain relevance scoring.	OpenSearch uses BM25 (TF-IDF variant). Score = sum of field scores. TF (term frequency): "headphone"
How to handle search schema changes?	Zero-downtime reindex: (1) Create new index with updated mapping (new-index), (2) POST /_reindex from
How to scale OpenSearch?	Scale reads: add replica shards (horizontal) — each replica handles read requests independently. Scale v